

Swing-Up and Balancing of a Furuta Pendulum via Deep Reinforcement Learning with Embedded Edge Inference

Aimilios Mavrothalassitis

Department of Electrical and Computer Engineering
Aristotle University of Thessaloniki

May 29, 2026

Abstract

This report covers the design, training, and physical deployment of a reinforcement learning (RL) controller for a custom-built Furuta (rotary inverted) pendulum. The mechanical structure is fabricated entirely from PLA via fused-deposition modelling (FDM) and designed in SolidWorks, except for aluminium support feet. A Proximal Policy Optimisation (PPO) agent is trained inside a physics simulation that uses domain randomisation, dynamic friction perturbations, and a gravity-tilt model to narrow the reality gap. The trained policy—a 3-layer MLP ($6 \rightarrow 64 \rightarrow 64 \rightarrow 1$, roughly 5.000 parameters)—is exported as a C++ header and runs at 50 Hz on a Teensy 4.0 microcontroller, with no external compute involved at runtime. The system manages autonomous swing-up and sustained balancing, and recovers reasonably well from manual pushes. Remaining limitations and ideas for follow-up work are discussed at the end.

Repository: github.com/EmiliosBlacksea/furuta-pendulum-rl

Contents

1	Introduction	2
2	Hardware Design	2
2.1	Mechanical Structure	2
2.2	Electronics	3
3	System Dynamics	5
3.1	Equations of Motion	5
4	Reinforcement Learning Formulation	5
4.1	Observation and Action	5
4.2	Reward Function	6
4.3	Policy and Training	6
5	Simulation and Sim-to-Real Transfer	7
5.1	Sensor Fidelity	7
5.2	Domain Randomisation	7
6	Embedded Firmware	8
6.1	Control Loop	8
6.2	Edge Inference	8
6.3	Dual Operating Modes	8
7	Experimental Results	9
7.1	Simulation Training	9
7.2	Real-Robot Performance	9
8	Future Work	10
9	Conclusion	10

1 Introduction

The Furuta pendulum is an underactuated mechanical system made up of a horizontal motor-driven arm with a freely rotating pendulum attached at its tip. The upright equilibrium is unstable by nature, which makes it a widely used benchmark for nonlinear control.

Classical approaches—LQR, energy-shaping controllers, and similar techniques—work reasonably well but require fairly accurate knowledge of the physical parameters and tend to break down when dynamics differ from the model. Reinforcement learning offers a different path: the agent figures out a control policy through trial and error, without needing an analytical model upfront, and can be made more tolerant of parameter uncertainty by training with randomised dynamics.

This project had four main goals:

1. Design and build a real Furuta pendulum using FDM 3D printing (PLA) on a SolidWorks-designed frame.
2. Train a PPO agent in simulation, including motor quantisation, velocity filtering, domain randomisation, and dynamic friction variation.
3. Deploy the trained policy as *edge inference* directly on a Teensy 4.0 microcontroller at 50 Hz, without a connected PC.
4. Build a serial protocol so the PC can drive inference, opening the door to online fine-tuning experiments later.

2 Hardware Design

2.1 Mechanical Structure

The frame and all moving parts were modelled in SolidWorks and printed on an FDM machine using PLA filament. The only non-printed parts are the aluminium support feet, which keep the base from flexing under load. Figure 1 shows the SolidWorks assembly.

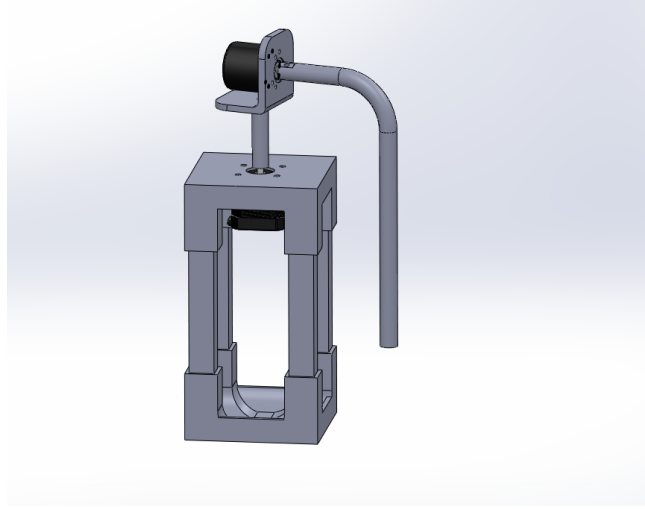


Figure 1: SolidWorks assembly of the Furuta pendulum.

The pendulum rod is a curved PLA part that rotates freely about the joint at the top of the main body. Its total mass is 48 g and its effective length, measured experimentally from the joint to the centre of mass, came out at $l = 0.1669$ m. The motor arm length, from the rotation axis to the joint, is $r = 0.12$ m.

2.2 Electronics

Table 1 lists the electronic components. Figure 2 shows the schematic captured in EasyEDA, and Figure 3 shows the high-level wiring layout.

Table 1: Electronic components

Component	Model	Role
Microcontroller	Teensy 4.0 (SparkFun DEV-15583), 600 MHz Cortex-M7	Control loop, edge inference
Stepper motor	Makerbase MKS SERVO42C NEMA17 (closed-loop)	Horizontal arm actuation
Encoder	Photoelectric incremental, 600 P/R (2400 CPR quad.)	Pendulum angle measurement
Power supply	MeanWell LRS-150-24, 24 V / 6.5 A	Motor power
Level shifter	CD74HCT125E quad buffer	$3.3\text{ V} \rightarrow 5\text{ V}$ for STEP/DIR/EN signals

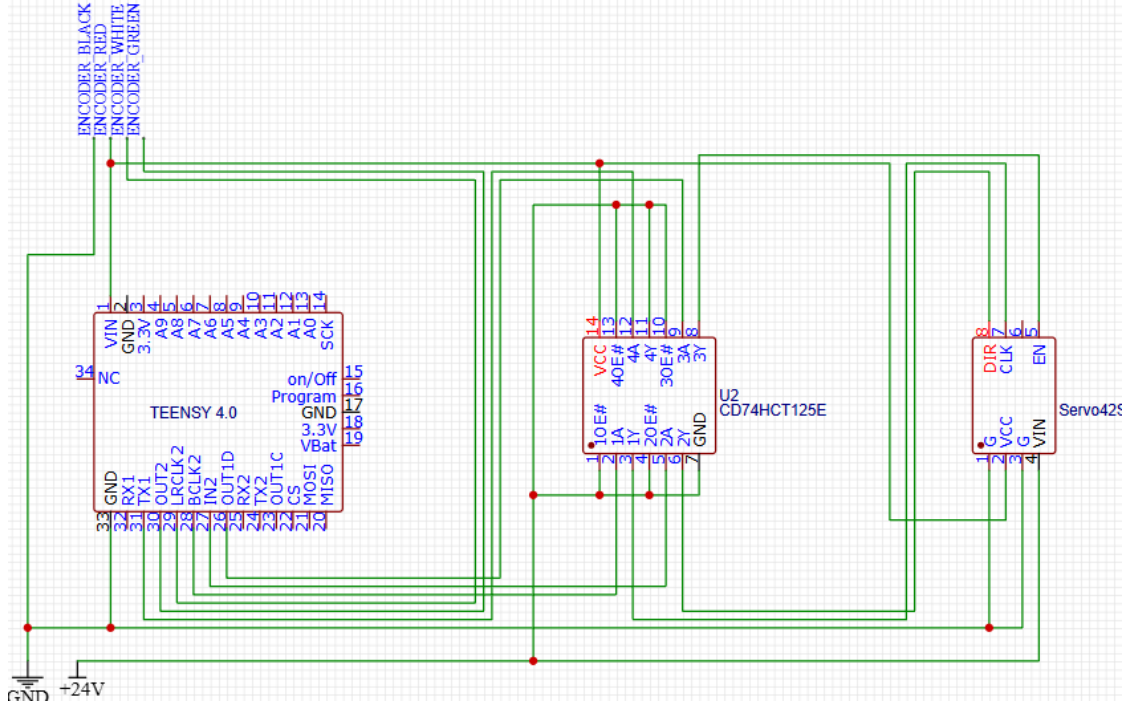


Figure 2: EasyEDA schematic. The Teensy 4.0 (left) drives the CD74HCT125E quad buffer (centre) which level-shifts the 3.3 V STEP, DIR, and EN signals to the 5 V logic required by the MKS SERVO42C (right). The encoder (top-left net labels) connects directly to 5 V-tolerant Teensy input pins. The MeanWell 24 V supply (bottom) powers the motor driver; logic is powered from the USB host.

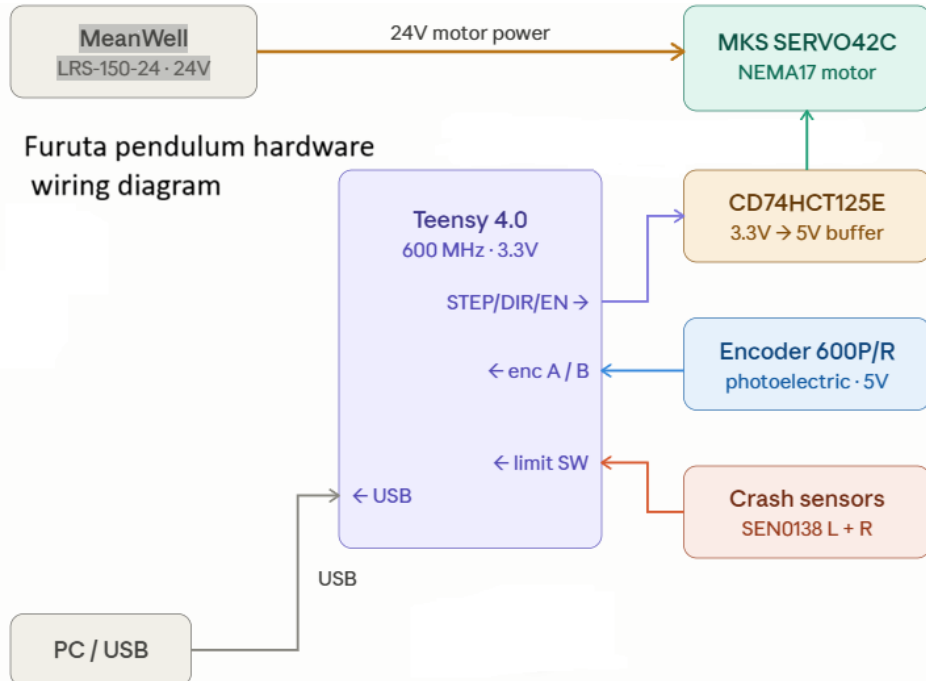


Figure 3: High-level wiring block diagram. Amber: 24 V motor power. Purple: 3.3 V control signals (Teensy → buffer). Teal: 5 V control signals (buffer → SERVO42C). Blue: encoder A/B feedback. Coral: limit switch digital inputs.

3 System Dynamics

3.1 Equations of Motion

Let ϕ denote the motor arm angle and θ the pendulum angle measured from the downward vertical ($\theta = 0$ down, $\theta = \pi$ upright). Applying the Lagrangian formulation gives the pendulum’s equation of motion:

$$(J + ml^2)\ddot{\theta} = -mgl \sin \theta - b\dot{\theta} + mlr\ddot{\phi} \cos \theta + ml^2\dot{\phi}^2 \sin \theta \cos \theta + w(t) \quad (1)$$

where $w(t)$ is a process disturbance injected during training for robustness. The motor arm is modelled as a first-order velocity lag:

$$\dot{\phi}(t) = \dot{\phi}^* + (\dot{\phi}(t^-) - \dot{\phi}^*)e^{-\Delta t/\tau} \quad (2)$$

with $\dot{\phi}^* = a \cdot v_{\max}$, where $a \in [-1, 1]$ is the RL action and τ the motor time constant.

Table 2: Physical parameters.

*Values marked † are experimental measurements; all others are nominal values used in the simulation and have not been independently identified on the physical system.

Symbol	Description	Value	Unit	
m	Pendulum mass (PLA part)	0.048	kg	†
l	Pendulum effective length	0.1669	m	†
r	Motor arm length	0.120	m	†
J	Pendulum moment of inertia	0.001336	kg m ²	nominal
b	Viscous damping coefficient	0.001	N m s	nominal
τ	Motor first-order time constant	0.00511	s	nominal
g	Gravitational acceleration	9.81	m s ⁻²	
$v_{\max}$	Maximum motor speed	3000	steps s ⁻¹	

4 Reinforcement Learning Formulation

4.1 Observation and Action

At each 20 ms timestep the agent receives a six-dimensional observation vector:

$$\mathbf{o}_t = [\phi_t, \dot{\phi}_t^{\text{flt}}, \cos \theta_t, \sin \theta_t, \dot{\theta}_t^{\text{flt}}, a_{t-1}^{\text{flt}}]^\top \in \mathbb{R}^6 \quad (3)$$

The $(\cos \theta, \sin \theta)$ pair avoids the discontinuity that would arise from using the raw angle at $\pm\pi$. Passing in the previous filtered action a_{t-1}^{flt} gives the policy a rough idea of the current motor velocity, which helps compensate for the partial observability introduced by the first-order motor lag.

The action $a_t \in [-1, 1]$ sets the motor velocity as a fraction of $v_{\max}$. An EMA filter smooths consecutive actions before they are applied:

$$a_t^{\text{filt}} = \alpha a_{t-1}^{\text{filt}} + (1 - \alpha) a_t, \quad \alpha = 0.3 \quad (4)$$

4.2 Reward Function

The reward uses the exponential-of-cost formulation, which keeps values bounded and differentiates well near the target:

$$r_t = \exp(-c_t), \quad c_t = \underbrace{1.0(1 + \cos \theta_t)}_{\text{upright}} + \underbrace{0.15|\phi_t|}_{\text{centre}} + \underbrace{0.02\dot{\theta}_t^2}_{\text{ang. speed}} + \underbrace{0.005|\dot{\phi}_t|}_{\text{smoothness}} + \underbrace{0.005(a_t - a_{t-1})^2}_{\text{jerk}} \quad (5)$$

The upright term is zero when $\theta = \pi$ and two at $\theta = 0$, so it acts as a continuous pull toward the goal. An episode ends if the motor arm goes beyond ± 4800 steps from centre (the physical rail limits), with a reward penalty of -1 .

4.3 Policy and Training

The policy network is a fully-connected MLP with roughly 5.000 parameters:

$$\mathbb{R}^6 \xrightarrow{\tanh} \mathbb{R}^{64} \xrightarrow{\tanh} \mathbb{R}^{64} \xrightarrow{\text{linear}} \mathbb{R}^1$$

Training uses Proximal Policy Optimisation via Stable-Baselines3. Table 3 lists the key hyperparameters.

Table 3: PPO hyperparameters

Parameter	Value
Learning rate	3×10^{-4}
Rollout steps	2048
Batch size	256
Discount γ	0.99
Entropy coefficient	0.005
Total timesteps	2.200.000
Obs. normalisation	VecNormalize (running mean/var, clip 10)

A curriculum callback gradually ramps sensor noise and domain-randomisation scale from zero up to their target values between steps 10.000 and 1.000.000. This lets the agent get a handle on the basic task before being hit with the full perturbations, which in practice made training considerably more stable.

5 Simulation and Sim-to-Real Transfer

5.1 Sensor Fidelity

The simulation (`SimMotorRLEnv`) tries to reproduce the following hardware characteristics as faithfully as practical:

Quantisation. Encoder ticks are rounded to the nearest $2\pi/2400$ radian (CPR = 2400) and motor steps to the nearest $2\pi/3200$ radian before velocities are computed, matching what the physical sensors actually produce.

EMA velocity filter. Velocities are estimated by finite difference over quantised positions and then filtered with an EMA (coefficient 0.80 in the simulation, corresponding to firmware constants `FILTER_ENC` = 0.35 and `FILTER_MOT` = 0.40). Getting these to line up precisely between simulation and hardware is still an open issue, discussed further in Section 8.

Action latency. A one-step communication delay is randomly inserted once the domain randomisation scale exceeds 0.5, to roughly model USB serial round-trip jitter.

5.2 Domain Randomisation

At every episode reset the following parameters are sampled independently:

Table 4: Domain randomisation ranges (at full scale)

Parameter	Range	Rationale
Mass m	$\pm 20\%$	Print mass variation
Pendulum length l	$\pm 20\%$	CoM estimation uncertainty
Inertia J	$\pm 35\%$	Geometry uncertainty
Damping b	$\pm 80\%$	Highly variable in practice
Motor time constant τ	$\pm 40\%$	Load-dependent
Gravity tilt	$\pm 4^\circ$	Frame levelling tolerance
Per-step friction mult.	$\times [0.2, 2.0]$	Joint friction variability

Random velocity kicks of up to 1 rad s^{-1} are applied to the pendulum at 0.5% probability per step, which turned out to be important for getting the physical robot to handle pushes gracefully.

6 Embedded Firmware

6.1 Control Loop

The Teensy runs a 50 Hz control loop organised as a four-state machine: IDLE, RUNNING, RESETTING, and WAITING. Each tick reads the encoder, computes EMA-filtered velocities, builds the six-dimensional observation vector, runs the MLP forward pass, applies the action filter, and drives the stepper via bit-banged STEP/DIR pulses.

6.2 Edge Inference

MLP weights live in flash as a C++ header (`model_weights.h`) generated by `scripts/export_weights.py`. At boot, `init_ram_weights()` copies them into RAM so they can be overwritten at runtime through the serial weight-streaming protocol. The forward pass uses 32-bit floating-point arithmetic, handled natively by the Cortex-M7 FPU, and comfortably fits within the 20 ms control budget.

6.3 Dual Operating Modes

STANDALONE. The Teensy runs the MLP on its own and streams telemetry (`TEL:steps,ticks`) to the PC dashboard. This is what was used for the demonstration.

PC_CONTROL. At each timestep the Teensy sends the observation vector to the PC (`OBS: $o_0, \dots, o_5$`), waits up to 15 ms for an action reply (`ACT: $a$`), and applies it. This turns the physical robot into a Gymnasium environment that can be used for online policy fine-tuning directly from real rollouts. The infrastructure for streaming updated weights back to the Teensy after each episode reset is in place, though evaluating how much it actually helps is left for future work.

7 Experimental Results

7.1 Simulation Training

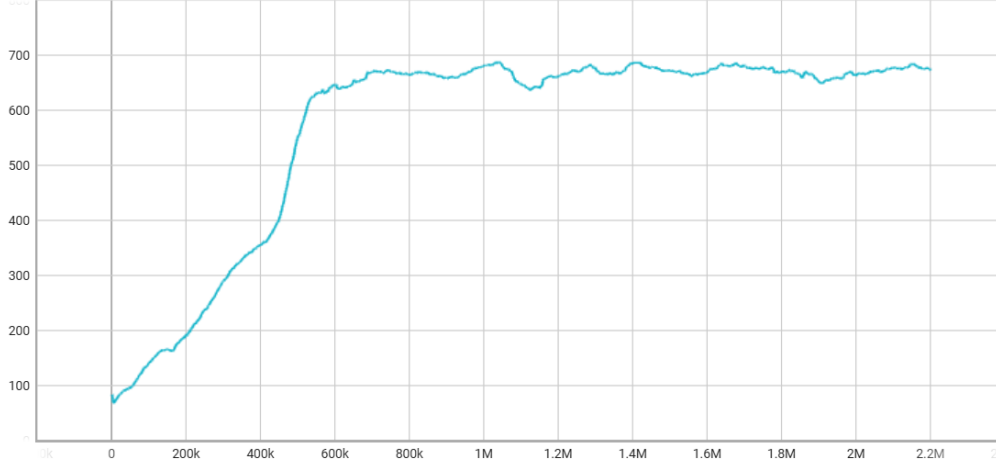


Figure 4: Mean evaluation reward vs. training steps (10-episode average). The dashed vertical line marks the point where full domain randomisation kicks in at step 1.000.000.

The agent converges to stable balancing in simulation while the curriculum brings in the full domain randomisation. TensorBoard curves show steady improvement in mean episode reward over 2.2 M steps; the checkpoint used on the physical robot was selected from evaluation episodes run under the full noise and DR configuration.

7.2 Real-Robot Performance

The policy was dropped onto the physical robot without any fine-tuning, straight from the best simulation checkpoint.

Swing-up. The agent reliably pumps the pendulum up from the downward rest position to the upright equilibrium within a few seconds.

Balancing and perturbation recovery. Once upright, the robot holds the position under normal conditions. After a manual push, the controller recovers and gets back to balance. That robustness was not explicitly engineered—it came out of the random kick perturbations included during training.

Motor arm drift. During extended balancing, the motor arm oscillates slowly around the centre. The drift is non-systematic: it changes direction and does not accumulate in one direction over time. The most likely cause is the mismatch between the EMA filter coefficients in the simulation and those in the firmware, which means the velocity

estimates the policy actually sees at runtime differ from what it was trained on. Fixing this is the main priority for the next iteration.

8 Future Work

System identification. The values for inertia J , damping b , and motor time constant τ are rough initial estimates and have not been measured on the actual build. Running proper identification experiments—step responses, sinusoidal frequency sweeps—would give a much tighter simulation model.

Fine-tuning evaluation. The online fine-tuning pipeline (PC_CONTROL mode, weight streaming, linear probing of the output layer) is built and working but has not been tested in any systematic way yet. The plan is to measure whether even a handful of real-robot episodes can reduce arm drift and improve balance duration.

Filter coefficient alignment. As noted above, the EMA filter coefficients differ between simulation and firmware. Bringing these into agreement—or explicitly identifying the mismatch and compensating for it in the reward or observation normalisation—is likely to have the biggest single impact on behaviour.

Parameter uncertainty quantification. The domain randomisation ranges were picked by intuition. Actually measuring the distribution of physical parameters (damping under different loads, for example) would let the randomisation ranges be both tighter and more realistic.

9 Conclusion

This project put together a complete pipeline from simulation training to embedded deployment of a PPO controller on a custom 3D-printed Furuta pendulum, and it mostly worked on the first physical transfer—which was the more uncertain part of the whole thing.

The main outcomes were:

- A fully 3D-printed Furuta pendulum (PLA) designed in SolidWorks, with an aluminium base for rigidity.
- A simulation with encoder quantisation, dynamic friction perturbation, gravity-tilt randomisation, and action latency to keep the sim-to-real gap manageable.
- A lightweight MLP policy ($\approx 5,000$ parameters) running at 50 Hz on a Teensy 4.0 with no PC involved at runtime.

- Successful swing-up, autonomous balancing, and perturbation recovery on the physical robot.
- A serial weight-streaming protocol that makes online fine-tuning possible without reflashing the firmware.

The main unsolved problem is the motor arm drift, which is almost certainly tied to the filter coefficient mismatch between simulation and hardware. Sorting that out, running a formal system identification, and actually evaluating the fine-tuning loop are the natural next steps.